第 5 章上下文治理：Memory、CLAUDE.md 与 Compact 是预算制度

5.1 上下文一多，系统就容易产生一种低级幻觉

人一旦可以往上下文里不停塞东西，就很容易相信一个朴素的神话：信息越多，系统越聪明。这个神话听起来甚至有点合情合理。毕竟知道得多，总比知道得少强。可惜代理系统不是图书馆，模型也不是藏书管理员。上下文不是一个“存进去就算拥有”的仓库，它首先是一笔昂贵、易膨胀、还会自我污染的预算。

Claude Code 的源码在这件事上很不浪漫。它并没有把上下文设计成一个可以无限堆叠的记忆池，反而在很多地方反复提醒自己：该加载什么、该截断什么、什么东西要长期保留、什么东西只能短期摘要，都是运行时必须严肃治理的事。

所以这一章要讨论的是：Claude Code 怎样防止自己被记住的东西拖死。这件事和“记住更多”看起来相近，工程上却是两种制度。前者偏向收藏癖，后者才接近治理术。

5.2 `CLAUDE.md` 体系说明，长期指令不能和临场对话混在一起

Claude Code 在 `src/utils/claudemd.ts` 开头就把记忆层次说得很清楚。它把 instruction source 分成几层：

- managed memory，例如 `/etc/claude-code/CLAUDE.md`
- user memory，例如 `~/.claude/CLAUDE.md`
- project memory，例如项目根目录里的 `CLAUDE.md`、`.claude/CLAUDE.md`、`.claude/rules/*.md`
- local memory，例如 `CLAUDE.local.md`

而且这些文件会按优先级和目录距离加载。离当前工作目录越近的 project 规则，优先级越高；越偏向私有、越偏向本地的规则，越晚加载，因而越靠近模型的注意力前沿。

这件事特别要紧。因为它说明 Claude Code 从一开始就拒绝把“长期协作规则”和“本轮临时对话”混成一锅粥。团队规范、个人偏好、仓库约束，这些东西的寿命远长于某一轮用户消息；如果把它们全都塞进聊天记录里，系统就会在两个极端之间摇摆：要么每轮都重复注入，浪费上下文；要么靠模型自己回忆，迟早失手。

`claudemd.ts` 给出的答案，是把这些稳定规则做成可发现、可分层、可组合的持久指令系统。还有个细节很有意思：它支持 `@include`，并且只允许一大批明确列出的文本扩展名。这说明工程师除了追求 include 的便利，也在提防另一种常见事故：有人把二进制、巨型文档、甚至不该进 prompt 的东西糊里糊涂带进来了。

这是一种很像正经工程师的克制。系统会先问：“什么东西值得进入系统记忆，什么东西一旦进入就是污染。”

5.3 `MEMORY.md` 是索引，不是日记本

如果 `CLAUDE.md` 管的是规则层，那么 `memdir` 处理的就是另一类更细的长期记忆。`src/memdir/memdir.ts` 里有一段设计很值得反复看：`ENTRYPOINT_NAME` 被定义成 `MEMORY.md`，但这个文件并不被鼓励用来直接堆内容，它被定义为 index。

源码里写得很实在。`buildMemoryLines()` 明确告诉模型，保存 memory 是两步：

1. 把具体 memory 写进独立文件
2. 再在 `MEMORY.md` 里加一个一行指针

为什么这么麻烦？因为系统知道入口文件天然会被频繁加载，而频繁加载的东西一旦变胖，整套上下文就会被它慢慢拖成一个不好收拾的胖子。

这也是为什么 `memdir.ts` 里专门有 `MAX_ENTRYPOINT_LINES = 200` 和 `MAX_ENTRYPOINT_BYTES = 25_000`。超过了，系统会直接 `truncateEntrypointContent()`，并在结尾追加明确警告：只加载了一部分，请把细节移到 topic files。

这套做法特别像一个见过太多失控索引的人。它不相信大家会天然克制，所以把“入口必须短”做成硬约束。因为入口文件一旦既当目录又当正文，最后就既不是目录，也不是正文，只是一个谁都不愿再读第二遍的烂尾摘要。

从 Harness Engineering 的角度看，这里抽出来的原则非常清楚：长期记忆必须分成“入口”和“正文”。入口负责低成本寻址，正文负责高密度承载。把两者混为一谈，最终一定是入口失效，随后整套记忆系统退化成摆设。

5.4 Session memory 说明，短期连续性也不能靠聊天记录硬扛

只有长期 memory 还不够。代理系统真正难受的地方，常常在于“这轮之前我们到底做到哪一步了”。这是一次会话内部的连续性问题。

Claude Code 在 `src/services/SessionMemory/prompts.ts` 里专门给这件事建了一套模板。默认模板里有这些栏目：

- `Current State`
- `Task specification`
- `Files and Functions`
- `Workflow`
- `Errors & Corrections`
- `Codebase and System Documentation`
- `Learnings`
- `Key results`
- `Worklog`

你一看就知道，这不是给人抒情的。它关心的是：现在做到哪了，踩过什么坑，改过哪些文件，后面该接什么。更有意思的是更新 prompt 的语气。源码里明确要求：

- 只能用 Edit tool 更新 notes file
- 不要提 note-taking 这件事本身
- 不要改模板结构
- `Current State` 必须始终反映最近工作
- 每节都要信息密集，但要控制预算

这说明 session memory 在 Claude Code 里并非“另存一份聊天记录”，它会把当前会话萃取成一种可继续工作的操作说明书。它不求完整复刻对话，而求压缩出未来继续干活所必需的骨架。

这里有个极其工程化的细节。`prompts.ts` 里定义了 `MAX_SECTION_LENGTH = 2000` 和 `MAX_TOTAL_SESSION_MEMORY_TOKENS = 12000`。超过预算，系统不会夸你记得细，而是要求你 aggressively condense，尤其优先保留 `Current State` 和 `Errors & Corrections`。

这很能说明问题。真正成熟的系统，会把“为继续工作保留最有用的部分”当成美德。因为上下文预算是工作内存。工作内存的第一职责是可操作。

5.5 自动 compact 说明，上下文治理首先是预算治理

到这里，长期规则、持久 memory、session memory 都有了，但上下文还是会膨胀。于是 Claude Code 在 `src/services/compact/autoCompact.ts` 里进一步承认一个现实：不管你多会整理，只要对话够长，总会逼近窗口边缘。

`getEffectiveContextWindowSize()` 先把模型 context window 减去一笔保留给 summary 输出的预算。`MAX_OUTPUT_TOKENS_FOR_SUMMARY` 直接预留了 20,000 tokens。也就是说，系统先假定 compact 本身要花钱，绝不把窗口吃到只剩一口气时才想起求生。

接着 `getAutoCompactThreshold()` 又在有效窗口上再扣掉 `AUTOCOMPACT_BUFFER_TOKENS = 13_000`。警告阈值、错误阈值、手动 compact 预留空间，也都各自分出 buffer。

这套数字背后有个很朴素的道理：上下文治理需要提前为失败和恢复留出余地。不留余地的系统，平时看着像节俭，出事时就暴露出本质，那只是把风险账单留给下一轮。

更有意思的是 `AutoCompactTrackingState`。它不仅记 `compacted`，还记 `turnCounter`、`turnId` 和 `consecutiveFailures`。这说明 autocompact 是一段会被追踪、会失败、会被限流的运行时行为。

源码甚至写了一个很直白的注释：全球每天曾经浪费大量 API calls 在连续失败的 autocompact 上，所以 `MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3`，再失败就触发 circuit breaker。这里的气质非常好，像一个终于受不了浪费的人：你可以失败，但不能无限次、无记忆地失败。

5.6 `compactConversation()` 说明，摘要要重建可继续工作的上下文

很多人一听 compact，会以为就是“把前面聊天摘要一下”。Claude Code 的实现要复杂得多。`src/services/compact/compact.ts` 里的 `compactConversation()` 真正做的，是把原有上下文拆开、摘要、再注入必要附件，重新搭出一个还能工作的后 compact 世界。

先看压缩前的清洗。`stripImagesFromMessages()` 会把图片、文档替成 `[image]`、`[document]` 之类的标记；`stripReinjectedAttachments()` 会把反正之后还要重新注入的 attachment 先剥掉，免得浪费 token。仅这两个动作就说明，compact 会有选择地丢掉那些“对于摘要无用、但对于 token 极贵”的部分。

再看摘要失败时的处理。源码里有 `truncateHeadForPTLRetry()`,专门应对“compact 请求自己都 prompt too long”的尴尬场面。也就是说，系统不仅承认主流程会爆，还承认“救火工具本身也会爆”。这很像真实世界，而不是 demo。

而在 compact 成功之后，Claude Code 做的不是简单保留一条 summary。它还会：

- 清空旧的 `readFileState`
- 重新生成 post-compact file attachments
- 把 plan attachment 补回来
- 把 plan mode attachment 补回来
- 把 invoked skills attachment 补回来
- 把 deferred tools、agent listing、MCP instructions 的 delta attachment 重新补回来
- 执行 session start hooks 和 post-compact hooks
- 写 compact boundary message，记录 pre-compact token 数与边界信息

这些动作合在一起，意思很明确：compact 的目标是把“继续干活所需的运行时环境”重新铺平。摘要只是中间产物，不是最终目的。

所以 compact 在 Claude Code 里更像一次受控重启，而不是一次聊天总结。旧上下文会被转译成新的工作底座。这种设计很值得记住，因为很多系统只做前半截，结果 compact 之后虽然“还记得大概”，却已经失去了工具状态、计划状态、附件状态，接下来还得再花几轮找回自己。

5.7 上下文治理的关键是保留工作语义

如果只看 `compact.ts` 的后半段，会发现一个贯穿始终的倾向：Claude Code 真正在意的是把工作语义保住。

例如它会恢复最近访问文件的 attachment，因为这些文件往往构成当前工作面的局部现实；它会恢复 plan mode，因为否则模型压缩完以后可能忘了自己还处在 plan discipline 里；它会保留 invoked skills 的内容，但又给每个 skill 设置 token cap，避免 skill 本身在 post-compact 阶段反客为主。

源码里这句话很有味道：per-skill truncation beats dropping。意思是，即使要裁，也优先保住开头那一段最关键指令，而不是整个扔掉。这就是治理，不是纯粹节流。纯节流是砍，治理是知道该砍哪里、该保什么。

从这里可以抽出一个相当稳妥的经验：上下文系统应该优先保留能维持行动语义的东西，而不是优先保留看起来信息量最大的东西。文件细节、当前计划、错误修正、技能

约束，这些都直接决定下一步能不能做对。反过来，冗长的历史对话、重复出现的附件、已经可由运行时重新发现的东西，就没必要再占着座位。

5.8 从源码里可以提炼出的第五个原则

这一章最后可以压成一句话：

> 上下文是工作内存。治理它的目标是支持系统继续工作。

Claude Code 的源码在几个层面共同支持这个判断：

- `claudemd.ts` 把长期指令分层加载，说明稳定规则要和临时对话分开治理
- `memdir.ts` 把 `MEMORY.md` 定义成索引并强行截断，说明入口文件必须短而可寻址
- `SessionMemory/prompts.ts` 用固定模板提炼会话连续性，并对 section 和总量设预算，说明短期记忆也必须结构化
- `autoCompact.ts` 为 compact 预留输出预算、缓冲区和失败熔断，说明上下文窗口要按风险来经营
- `compact.ts` 在摘要后恢复计划、文件、技能、工具附件和 hook 状态，说明 compact 的目标是重建工作语义，而不是写一段好看的总结

如果把这些抽象成可迁移的工程原则，大概有这样几条：

- 长期规则、长期记忆、会话连续性，应该分层，不该混写
- 入口型记忆必须短小，否则整个系统会被入口拖垮
- session summary 应该服务于“继续工作”，而不是服务于“回忆完整”
- compact 是上下文治理主路径
- 压缩后的上下文必须保住运行语义，而不是只保住语言表面

下一章要讲的是这套治理系统碰到极限时怎么办。因为一个真系统终究会出错：prompt too long，max output tokens，hook 死循环，恢复分支相互打架。到那时你才看得出来，一个代理系统到底是在“赌不出事”，还是在认真设计出事之后怎么活下去。

第 6 章错误与恢复：出错后仍能继续工作的代理系统

6.1 工程世界最不值得相信的话，就是“正常情况下”

很多系统设计文档里，最常见的偷懒方式，就是先讲一遍“正常情况下”的流程，仿佛只要主路径足够漂亮，错误就会自动显得次要。可代理系统一旦进入真实运行环境，这种写法通常很快就会露馅。因为模型会被截断，请求会超长，hook 会制造回环，工具会中断，fallback 会发生，恢复逻辑本身也会失手。

所以，判断一个代理系统成熟不成熟，不能只看它回答顺畅的时候有多像个人，而要看它出故障的时候像不像系统。前者容易靠一点 prompt 工程粉饰，后者只能靠运行时纪律。

Claude Code 在这一点上的可取之处，是它没有假装自己不会出错。相反，源码里反复体现出一种冷静判断：错误属于主路径，恢复则是必须提前设计好的运行机制。

6.2 `prompt too long` 是一种必然周期

对长会话代理来说，`prompt too long` 是一种迟早会来的季节变化。你如果把它当偶发异常，系统迟早会被它教育。

Claude Code 的 `query.ts` 就没有把它当偶发异常处理。在 `src/query.ts` 里，系统甚至会“暂时扣下”这类错误，不立刻把它原样抛给用户。流式阶段里，`withheld` 逻辑会识别 recoverable errors，包括：

- prompt too long
- media size error
- max output tokens

这件事的意思很明确：有些错误要先交给恢复系统试着处理，再决定是否展示给用户。这个顺序很关键，因为用户真正关心的通常是系统还能不能继续干活。

在 prompt too long 的分支上，Claude Code 先尝试更便宜、更保守的恢复路径。若启用了 context collapse，就先 `recoverFromOverflow()`，把已经 staged 的 collapse 提交掉；如果还不行，再进入 `reactiveCompact.tryReactiveCompact()`。也就是说，恢复是分层的：先排空已知积压，再做更重的全文压缩，不会一上来就重建世界。

这种次序特别有工程味。因为真正好的恢复系统，不会把所有错误都交给“最重的一把锤子”。它会先试图保住最细粒度的上下文，再在必要时接受更粗糙的摘要替代。

6.3 响应式 compact 说明，恢复的关键在于别把自己逼进死循环

很多系统做恢复时容易犯一个愚蠢但常见的错误：一旦发现错误可恢复，就不停重试，直到把错误从偶发事件升级成资源灾难。

Claude Code 对这件事非常警惕。`query.ts` 里有两个地方都能看出这种警惕。

第一处，是 `hasAttemptedReactiveCompact`。一旦 reactive compact 已经试过，再次遇到同类问题时，系统不会装傻重来。因为工程师很明白：如果 compact 之后还是不行，那么继续 compact 大概率只是在把同一种失败换个姿势再演一次。

第二处，是 stop hooks 的防死循环处理。源码里有非常直白的注释：如果 prompt too long 之后还让 stop hooks 介入，就可能出现 death spiral，路径大致是：

错误 -> hook blocking -> retry -> 错误 -> hook blocking

这话写得一点也不文学，却比许多文学都诚实。因为它承认系统里最危险的错误，是失败分支和恢复分支彼此咬住，开始无限自我复制。

所以 Claude Code 在 prompt too long 无法恢复时，会直接 surface 错误并跳过 stop hooks。原因很简单：这时候继续走形式流程，只会让坏事更有仪式感。

6.4 `max_output_tokens` 的处理说明，恢复要以续写为主

大模型产品有个很坏的习惯，一旦输出截断，就先来一段很客气的废话：抱歉，刚才被截断了，我来总结一下。听上去态度不错，实际上对工作几乎没帮助。

Claude Code 在 `src/query.ts:1185` 往后的处理，明显更接近工程系统。它先试一种成本较低的恢复：如果当前使用的是较保守 cap，就把 `maxOutputTokensOverride`

提升到更高值，直接重跑同一请求。注意，这一步没有插入 meta message，也没有让模型先寒暄，系统先给它一次把原任务做完的机会。

如果更高 cap 也不够，再进入第二层恢复：给模型追加一条 meta user message，内容非常实在，大意是：

直接继续，不要道歉，不要 recap，若中断发生在半句，就从半句接着写；剩余工作拆小一点。

这是一条很有启发意义的系统指令。它说明 Claude Code 对恢复的理解，是尽量保持任务连续性，不要把额外 token 花在体面动作上。在长任务里，这种区别非常大。因为每一次截断后的 recap，都会进一步消耗预算，并且增加语义漂移。最后系统做的就不再是任务本身，而是一轮轮地回顾自己做任务。

所以对 `max_output_tokens` 来说，较好的恢复通常是续写。Claude Code 在这里优先保证任务连续性，而不是补充礼貌性说明。

6.5 auto compact 的失败熔断，说明恢复系统自己也要受治理

如果说前面讲的是“单次错误怎么救”，那 `src/services/compact/autoCompact.ts` 处理的就是另一个层面的问题：当恢复机制本身不断失败时，怎么办。

源码的回答很简单，也很正确：别一直试。

`AutoCompactTrackingState` 里专门有 `consecutiveFailures`。一旦失败次数超过阈值，`shouldAutoCompact` 即便判断“按理说该 compact 了”，系统也会直接跳过。源码注释甚至给出过往数据：曾有大量 session 在连续 autocompact failure 上白白烧掉海量 API calls，所以必须加 circuit breaker。

这里特别值得强调的一点是，失败熔断意味着承认当前恢复手段在这个局面里已经失效。一个真正成熟的系统，不能只会在成功时记录指标，也得在失败时懂得收手。不会收手的恢复系统，和不会刹车的汽车差不多，理论上都叫系统，实际上都不该上路。

从 Harness Engineering 角度看，这里可以抽出一条很硬的原则：任何自动恢复机制都必须可计数、可限次、可熔断。否则恢复会从保险丝变成新的起火点。

6.6 compact 自己也会爆，所以连“修复动作”都需要修复策略

`compactConversation()` 这段代码还有一个很动人的现实主义时刻：它承认 compact 请求自己也可能 `prompt too long`。

这件事看上去带一点黑色幽默。系统为了缩短上下文去发摘要请求，结果摘要请求也因为上下文太长而失败。很多人不喜欢这种情形，因为它暴露得过于直接。但工程系统首先要解决的是继续运行，而不是保持表面完整。

Claude Code 的处理方式，是在 `compact.ts` 里引入 `truncateHeadForPTLRetry()`。当 compact 自己太长时，系统会先把更早的 API round 成组地从头部剥掉，再重试 compact，避免让用户卡在“连压缩都压不动”的状态里。

这里的取舍很清楚：这种修复有损，也会丢历史，但它优先保证用户不被完全锁死。源码注释写得很实在，这是一种 last-resort escape hatch。

这种处理方式的价值，在于它没有回避现实约束。系统快要窒息时，优先级是先恢复呼吸，再讨论信息保真度。这个判断不追求漂亮，但很实用。

6.7 abort 语义说明，中断也属于错误恢复的一部分

很多人把 abort 单独归到交互体验，不愿意把它放进错误恢复讨论里。从运行时角度看，中断就是一种必须被正确回收的失败态。

Claude Code 在两个层面都认真处理了这件事。

一层在 `query.ts`。如果 streaming 时用户打断，系统会先消费 `StreamingToolExecutor.getRemainingResults()`，为已经发出但尚未完成的工具生成 synthetic tool_result，确保前面承诺过的 `tool_use` 不会变成悬空债务。

另一层在 `compact.ts`。源码里专门把 compact 的 abort controller 传给 forked agent，并处理 `APIUserAbortError`，防止“被用户按了 Esc 的 compact”误算成一次成功摘要。

这两处连在一起看，意思很明确：中断不只是“用户不想看了”，而是一次需要正确收尾的状态转移。错误恢复如果只管异常、不管中断，最终会留下大量语义半残的执行轨迹。那种轨迹通常短期内没人查，长期看全是祸根。

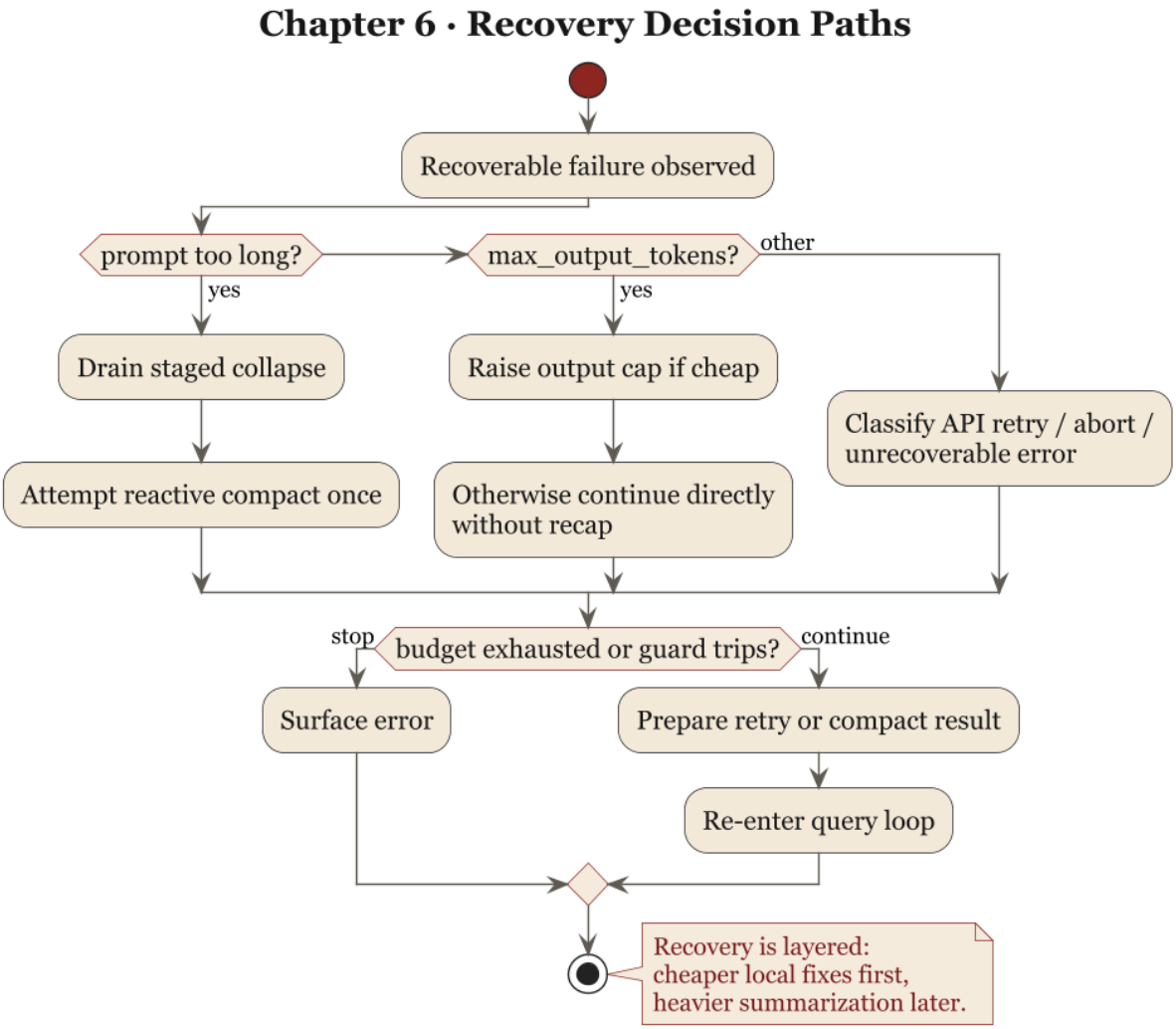

图 6: Claude Code Recovery Decision Paths

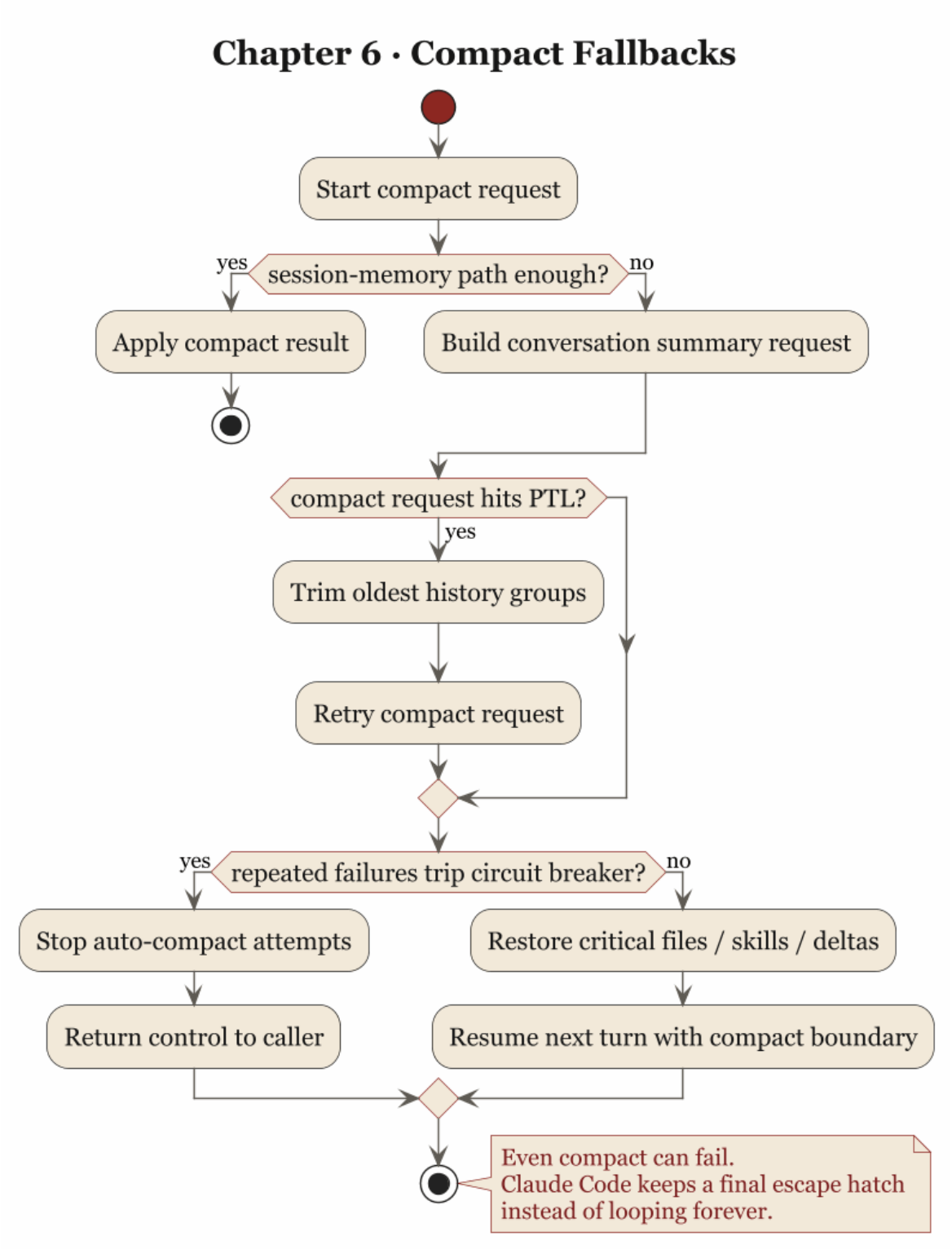

图 7: Claude Code Compact Fallbacks

6.8 错误处理真正保护的，是执行叙事的一致性

把这些源码放在一起看，会发现 Claude Code 对错误与恢复的理解，有一个很核心但常被忽略的目标：保护执行叙事的一致性。

什么叫执行叙事？很简单，就是系统还能不能说清楚：

- 我刚才试图做什么
- 为什么没做成
- 我用了什么恢复路径
- 现在是继续、停止，还是换轨

`query.ts` 里的 `transition.reason`，`maxOutputTokensRecoveryCount`，`hasAttemptedReactiveCompact`，以及 compact boundary、synthetic error message 这些东西，都是为了让这条叙事线不断裂。它们是为了让系统自己别失忆。

一个没有叙事一致性的代理系统，表面也许还能继续输出，但它的内部已经开始散了。今天用户看到的是多说几句废话，明天运维看到的就是为什么这里一会儿是 hook retry，一会儿是 compact retry，后天团队看到的则是这个系统出了问题谁都说不清它到底经历了什么。

所以错误恢复真正修补的，不只是错误本身，还有系统对自己行为的解释能力。解释能力一断，系统就会从工程对象退化成玄学对象。

6.9 从源码里可以提炼出的第六个原则

这一章最后可以收成一句话：

> 代理系统的体面，体现在错误发生后仍能维持可解释、可限界、可继续的执行秩序。

Claude Code 的源码在几个点上共同支持这个判断：

- `query.ts` 会暂时扣下可恢复错误，先交给恢复分支处理，说明错误要先尝试转化
- prompt-too-long 恢复先走 collapse drain，再走 reactive compact，说明恢复路径按成本和破坏性分层
- `hasAttemptedReactiveCompact` 与 stop hook guard 明确防止死循环，说明恢复本身也要受治理

- `max_output_tokens` 先提 cap，再要求模型直接续写，说明恢复的目标是延续任务，不是补充礼貌动作
- `autoCompact.ts` 的 consecutive failure 与 circuit breaker，说明自动恢复必须可熔断
- `compact.ts` 对 compaction 自身的 prompt-too-long 也有降级修复，说明连修复动作本身都要有恢复策略

如果把这些抽成可迁移的工程原则，大概是这样：

- 错误恢复要分层，不要所有问题都打一把重锤
- 恢复逻辑必须防止自我回环
- 自动恢复需要计数和熔断
- 截断后的最佳恢复通常是续写，不是总结
- 中断也是一种需要语义收尾的失败态
- 一个系统是否可靠，最终要看它出错后还能不能把自己的行为讲明白

下一章要进入另一类更棘手的问题：多代理与验证。因为当一个系统不再只是“自己出错自己救”，而开始把任务分给别的 agent，再把结果收回来复核，错误与恢复的问题就从单线程秩序升级成了组织问题。那时你面对的不只是一个模型会不会失手，而是一群不稳定执行体如何彼此约束。